

Rapport du projet: **P.G.L Voronoï/Delaunay**

1. Introduction

a) Attendu du projet

Le but de notre projet est de créer une application possédant une interface graphique permettant de gérer un partitionnement spatial 2D d'une carte.

Les différents attendus de notre projet sont :

- La gestion des points de références et des utilisateurs (ajout, suppression, déplacement).
- L'affichage d'un lien direct entre un utilisateur et un point de référence.
- l'ajout en masse d'utilisateurs avec des positions aléatoires.
- La visualisation des zones de Voronoï avec l'affichage des propriétés des points de références.
- L'importation en masse des points de référence depuis un fichier (format libre).
- L'importation et l'exportation en binaire obligatoirement de la carte complète.
- L'affichage des informations concernant la triangulation.
- une interface graphique avec JavaFX.
- une version en ligne de commande

b) périmètre Éthique et Design

Lors de nos travaux en éthique et design, nous avons décidé de partir sur le sujet d'une application de gestion des hôpitaux. Les utilisateurs pourront donc chercher l'hôpital le plus proche de sa position. Plusieurs données sont prises en compte telles que la saturation des hôpitaux, auquel cas l'utilisateur devra être redirigé vers le prochain hôpital.

Dans notre contexte, les points de références représentent donc les hôpitaux.

2. Organisation et flux de travail

Ce projet a été réalisé dans le cadre d'un travail de groupe, notre équipe était composée de 5 étudiants nous avons utilisé un dépôt git pour stocker notre projet et avons utilisé WhatsApp pour communiquer.

Pour ne pas se perdre et avoir à peu près la même chose sur chaque machine, nous avons opté pour utiliser VSCode comme environnement de développement. Nous avons utilisé Maven pour gérer les dépendances du projet (JavaFX, JUnit) et pour compiler et lancer l'application de manière uniforme sur toutes les machines avec les commandes `mvn compile` et `mvn javafx:run`.

Au début du projet, nous avons décidé de créer une checklist pour nous aider à organiser notre travail. Chacun devait cocher et indiquer ce qu'ils allaient faire pour que 2 personnes ne fassent pas la même chose et donc perdre du temps.

Nous avons donc commencé par créer le package Model qui contient toutes les classes utiles à notre projet comme Point, User qui représente la classe des utilisateurs, Hospital celle des hôpitaux, HospitalZone une zone de voronoï, et la carte VoronoiMap.

Ensuite, nous avons réfléchi à comment calculer les zones de voronoï et avons conclu que la triangulation de delaunay avec l'algorithme de bowyer Watson serait le plus simple à implémenter. Nous avons donc rajouté une classe Triangle.

Nous nous sommes donc informés sur comment cet algorithme fonctionne pour pouvoir l'implémenter.

Pour répondre aussi à notre sujet en design et éthique, nous avons implémenté une logique de redirection des utilisateurs lorsque l'hôpital le plus proche est saturé.

Nous avons créé une classe VoronoiEngineStub pour pouvoir séparer notre équipe : Une partie fait la version ligne de commande et l'autre l'interface graphique mais finalement nous avons décidé de nous concentrer d'abord sur la version ligne de commande, donc cette classe ne nous a pas servi à grand chose.

Nous avons donc créé un package Cli contenant les classes des commandes d'import/export de fichier, de gestion des utilisateurs et celle des hôpitaux.

C'est la classe CommandLineMain qu'on doit exécuter pour avoir la ligne de commande.

Enfin nous sommes passés sur la partie interface graphique avec SceneBuilder où nous avons construit les éléments de notre interface. Puis nous avons créé le MapController qui interagit directement avec sceneBuilder grâce au `fx:id`, le mapCanvas qui dessine les éléments graphiques sur l'interface graphique, comme les points, les liens (représenté par des segments), la couleur lorsqu'un élément est sélectionné etc... et enfin nous avons la classe MainApp qui est le point d'entrée, c'est elle qui démarre l'application javaFX et qui lance donc la fenêtre.

Chaque semaine, nous organisons une réunion avec notre tuteur afin de faire un point sur l'avancement du projet. Ces réunions nous permettaient de présenter ce que nous avons réalisé. Elles nous ont aidées à rester dans la bonne direction et à ne pas nous éloigner des exigences du cahier des charges. Ces points réguliers ont également été l'occasion de valider ou revoir nos choix de conception avant de continuer le développement.

3. Problèmes rencontrés et solutions trouvés

a) Conflits Git

Au début du projet, notre équipe a rencontré plusieurs problèmes liés à l'utilisation de Git. On oubliait de récupérer les modifications des autres avant de commencer. De plus, l'utilisation fréquente au début de git push --force écrasait le travail des autres, nous avons donc interdit l'usage. Certains membres ont opté pour la création d'une branche personnelle.

b) Mauvaise modélisation de la classe HospitalZone

Lors de la phase de conception orientée objet, nous avons initialement créé une classe HospitalZone héritant directement de la classe Hospital. Après réflexion, cette modélisation s'est révélée incorrecte. En effet, un hôpital représente un point géographique précis, tandis qu'une zone hospitalière correspond à une surface géographique plus large. Une zone n'est donc pas un type particulier d'hôpital mais un concept distinct. Pour corriger cela nous avons donc modifier la relation entre ces deux classe, HospitalZone possède désormais un attribut centre qui est l'hôpital associé à la zone.

c) Problème d'affichages des zones de voronoï sur la interface

Lors de l'affichage des zones hospitalières sur la carte, nous avons constaté plusieurs anomalies visuelles. Certaines zones présentaient des formes incomplètes. Pour corriger ce problème, nous avons ajouté des « points fantômes » sur les bords de la carte. Ces points supplémentaires permettent aux algorithmes de génération des zones de calculer correctement les frontières et d'obtenir un rendu plus naturel. Après cette modification, l'affichage des zones est devenu beaucoup plus cohérent et visuellement satisfaisant.

d) Trie de la liste des utilisateurs

Dans une première version de l'application, les hôpitaux étaient triés à l'aide de l'algorithme Bubble Sort afin de maintenir une liste ordonnée pour chaque utilisateur. Bien que simple à implémenter, cet algorithme possède une complexité temporelle élevée, ce qui entraîne des performances médiocres lorsque le nombre d'hôpitaux augmente.

Afin d'améliorer l'efficacité du système, nous avons remplacé cette approche par l'utilisation d'un Comparator associé aux méthodes de tri fournies par les bibliothèques standard de Java. Cette solution permet d'obtenir un code plus propre, plus maintenable et surtout beaucoup plus performant.

e) Séparation des responsabilités dans le package Algorithm

Lors de la conception du package Algorithm, nous avons regroupé dans une seule classe TriangulationDelaunay à la fois la gestion de la carte (ajout, suppression, déplacement des hôpitaux et utilisateurs) et le calcul mathématique pur de l'algorithme de Bowyer-Watson (triangulation, construction des zones Voronoï). Après réflexion, nous avons réalisé que cette conception ne respectait pas le principe de responsabilité unique : chaque classe

devrait avoir une seule raison de changer. Idéalement, il aurait fallu séparer ces deux responsabilités en deux classes distinctes BowyerWatson: uniquement le calcul mathématique et TriangulationDelaunay : uniquement la gestion de la carte, déléguant les calculs à BowyerWatson. Par manque de temps, nous n'avons pas pu effectuer cette refactorisation complète. Cependant cette prise de conscience nous a appris l'importance de bien définir les responsabilités de chaque classe dès la phase de conception, avant de commencer à coder.

4. Résultat

Notre application finale répond à peu près à l'ensemble des exigences du cahier des charges. Elle permet de gérer une carte hospitalière interactive avec les fonctionnalités suivantes :

Interface graphique

- Ajout, suppression et déplacement d'hôpitaux et d'utilisateurs manuellement
- Ajout en masse d'utilisateurs à positions aléatoires
- Affichage des zones Voronoï colorées par hôpital
- Affichage de la triangulation de Delaunay
- Affichage des liens entre utilisateurs et hôpitaux assignés
- Affichage de la redirection des utilisateurs
- Sélection d'un hôpital avec affichage des statistiques (saturation, capacité, redirigés, surface de zone)
- Sélection d'un utilisateur avec affichage de l'hôpital le plus proche et de l'hôpital assigné
- Zoom et déplacement de la vue
- Import en masse d'hôpitaux depuis un fichier CSV
- Export et import de la carte complète en format binaire
- Effacer

Nous n'avons pas eu le temps d'afficher les informations sur les triangles lorsqu'on clique dessus .

Version ligne de commande

- Toutes les opérations de gestion des hôpitaux et utilisateurs
- Import CSV et import/export binaire
- Affichage des triangles Delaunay et des statistiques

Logique de redirection

Lorsqu'un hôpital est saturé, l'utilisateur est automatiquement redirigé vers le prochain hôpital disponible le plus proche Les utilisateurs redirigés sont affichés en rouge sur la carte.

Diagramme de Classe

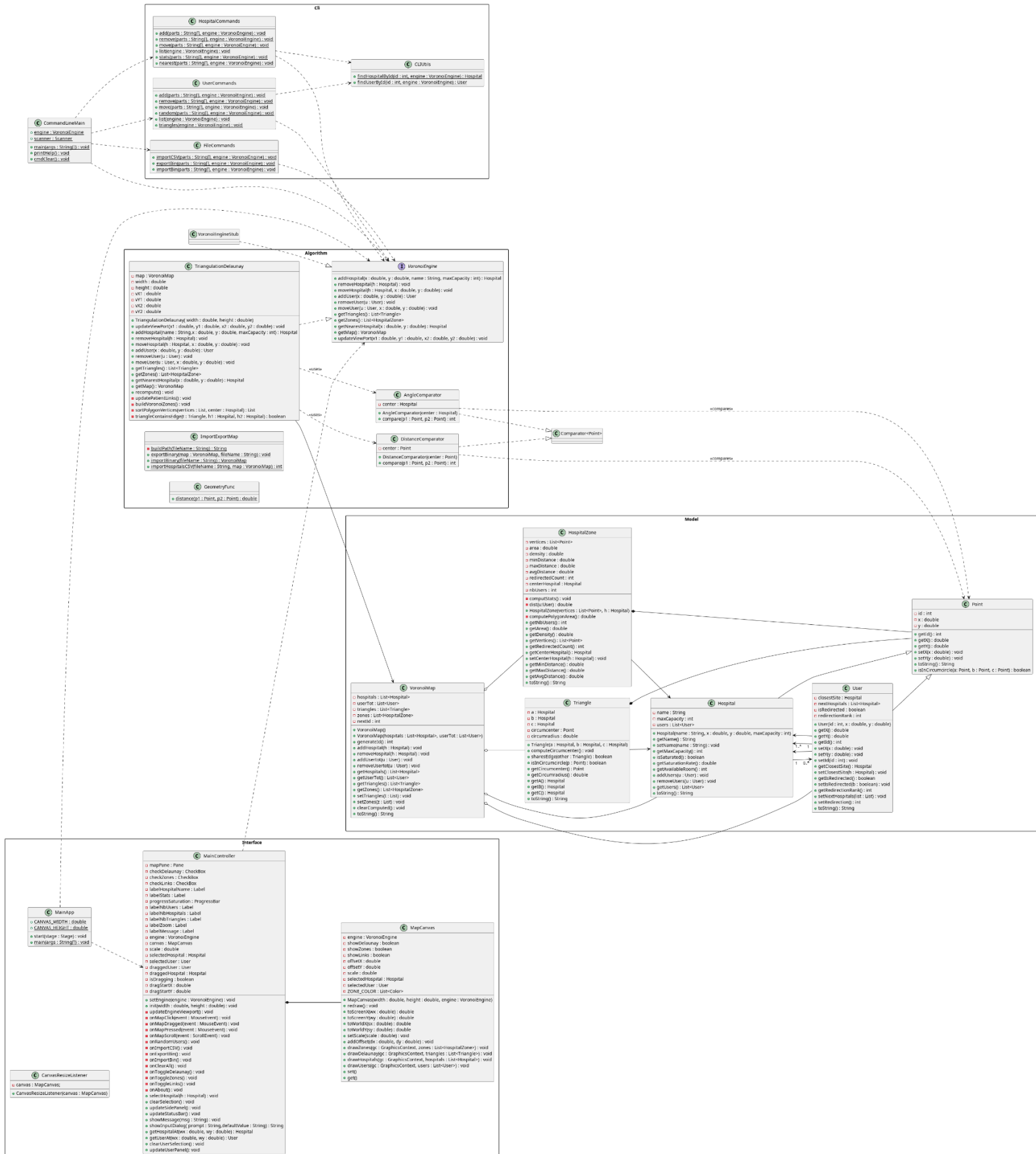


Diagramme de cas d'utilisation

